Migration Plan: `ams_np` Precise Dynamics → `mpc_ef` Acados MPC

1. Executive Summary

Replace the simplified decoupled dynamics in `mpc_ef` with the full coupled Newton-Euler dynamics from `ams_np`. This makes the MPC prediction model match the real physics — eliminating model mismatch that currently degrades tracking accuracy (especially during fast arm motions or aggressive maneuvers).

---

2. Current Architecture Comparison

2.1 `mpc_ef` (simplified, current)

Aspect	Detail
Base dynamics	Rigid body: thrust along body-z only, diagonal inertia Euler eqs
Arm dynamics	**Decoupled** double integrator: `ddq` as direct control input
Coupling	**None** — arm motions do not affect base and vice versa
FK	2D planar FK in body XZ plane (custom, with −π joint-2 offset)
Parameters	`m=2.1 kg`, `I=diag(0.03,0.03,0.05)`, `L1=L2=0.2 m`
Controls (6)	`[thrust, τ_x, τ_y, τ_z, ddq1, ddq2]`
State (17)	`[pos(3), vel(3), quat(4), ω(3), q_arm(2), dq_arm(2)]`
Symbolic framework	CasADi (required by acados)

2.2 `ams_np` (precise, target)

Aspect	Detail
Base dynamics	Full Newton-Euler: `F_ext` and `τ_ext` include arm reaction forces
Arm dynamics	**Coupled** via mass matrix: `M(q) * qddot = u − h(q,qdot)`
Coupling	**Full** — arm Coriolis/centrifugal/gravity forces feed back into base
FK	3D DH transforms with mount rotation matrix
Parameters	`m_platform=1.5 kg`, `m_link1=0.15 kg`, `m_link2=0.12 kg`, `L1=0.25 m`, `L2=0.20 m`, full 3×3 inertia matrices
Controls (8)	`[F_ext(3), τ_ext(3), τ_joint1, τ_joint2]`
State (17)	Same layout: `[pos(3), vel(3), quat(4), ω(3), θ(2), θ̇(2)]`

Aspect	Detail
Symbolic framework	Pure NumPy (no CasADi)

2.3 Key Differences at a Glance

```
┌──────────────────────┬──────────────────────────────┬──────────────────────────────┐
┐
│                      │  mpc_ef (simplified)         │  ams_np (precise)
│
├──────────────────────┼──────────────────────────────┼──────────────────────────────┤
┤
│ Arm–base coupling    │  None                        │  Full Newton-Euler
│
│ Link masses          │  Ignored (lumped)            │  m1=0.15, m2=0.12 kg
│
│ Link inertias        │  Ignored                     │  Full 3×3 per link
│
│ Coriolis/centrifugal │  Ignored                     │  Computed recursively
│
│ Gravity on arm       │  Ignored                     │  Per-link gravity terms
│
│ Arm control input    │  ddq (acceleration)          │  τ (torque)
│
│ Base force input     │  Scalar thrust (body z)      │  Full 3D force
│
│ FK convention        │  Planar XZ, custom           │  3D DH + mount rotation
│
│ Link lengths         │  L1=L2=0.20 m                │  L1=0.25, L2=0.20 m
│
│ CasADi support       │  Yes                         │  No (NumPy only)
│
└──────────────────────┴──────────────────────────────┴──────────────────────────────┘
┘
```

---

3. Migration Challenges

3.1 CasADi Symbolic Re-implementation (CRITICAL)

Acados requires CasADi symbolic expressions for `f_expl_expr`. The `ams_np` package is pure NumPy — loops over Python lists, `np.cross`, `np.linalg.solve`. None of these work symbolically.

Options:

Option	Effort	Runtime	Accuracy
(A) Fully symbolic Newton-Euler — Rewrite the recursive ID/FD in CasADi SX	High	Best (compiled)	Exact

Option	Effort	Runtime	Accuracy
(B) CasADi `callback` wrapping NumPy — Use `casadi.Callback` to call ams_np at each NLP iteration	Low	Slow (Python per call)	Exact
(C) Explicit symbolic closed-form — Manually expand the 2-link coupled EOM into closed-form CasADi expressions	Medium	Best (no loops)	Exact
(D) Hybrid: CasADi FK + explicit coupling terms — Keep base dynamics as-is, add coupling correction terms symbolically	Medium	Good	Approximate

Recommendation: Option (A) — Fully symbolic Newton-Euler in CasADi SX.

Although the effort is higher, Option (A) mirrors the proven `ams_np` code structure line-by-line. Since the recursion in `ams_np` already works and is well-tested, translating it to CasADi SX is a mechanical (non-creative) process: replace `np.cross` → `casadi.cross`, `np.array` → `casadi.vertcat`, etc. The resulting symbolic graph is equivalent to an unrolled closed-form but retains the readable recursive structure. It also scales to more joints without rewriting.

3.2 Control Input Mapping

Current `mpc_ef`: `u = [thrust, τ_x, τ_y, τ_z, ddq1, ddq2]` (6 controls)

`ams_np`: `u = [F_ext(3), τ_ext(3), τ_j1, τ_j2]` (8 controls)

Two sub-problems:

3.2.1 Base force: scalar thrust vs. full 3D force

The real quadrotor can only thrust along body z. `ams_np` accepts arbitrary `F_ext` because it's a general dynamics engine.

Decision: Keep **scalar thrust** as MPC control input (physically correct). In the dynamics expression, set `F_body = [0, 0, thrust]` and rotate to world frame, then feed into the coupled equations as `F_ext = R @ F_body + m_total * g`. This matches what the real platform can produce.

3.2.2 Arm: acceleration vs. torque

Currently the MPC outputs `ddq` directly and the arm dynamics is $\dot{\theta} = ddq$ (double integrator). With coupled dynamics, the relationship is:

```
M(q) * [a_A; α_A; θ̈]  =  [F_ext; τ_ext; τ_joint]  −  h(q, q̇)
```

Two approaches:

Approach	Pros	Cons

Approach	Pros	Cons
(i) Keep ddq as MPC input — Substitute $\ddot{\theta} = ddq$ into the coupled EOM and solve for the required τ_{joint} to achieve it. The MPC sees a "virtual" acceleration input.	Backward compatible with current MPC cost/reference setup; easy to warm-start	Requires computing M and h to reconstruct torques; hides actuator limits
(ii) Switch to torque input — Let MPC output τ_{joint} directly and let the dynamics compute the resulting $\ddot{\theta}$	Physically accurate; can enforce actuator torque limits directly	Changes control dimension; existing cost tuning needs re-work

Recommendation: Approach (ii) — switch to torque input.

Rationale: Since the whole point is precise physics, hiding the mass matrix behind a virtual acceleration input defeats the purpose. Torque limits can also be enforced. The control vector becomes:

```
u_new = [thrust, τ_x, τ_y, τ_z, τ_j1, τ_j2]   (still 6 controls)
```

Dimensionality stays the same — only the interpretation of the last two entries changes. The solver cost matrices and bounds need updating.

3.3 Parameter Synchronization

Parameter	`mpc_ef`	`ams_np`	Action
Total mass	2.1 kg (flat)	1.5 + 0.15 + 0.12 = 1.77 kg	Use `ams_np` values; update `HOVER_THRUST`
Platform inertia	diag(0.03, 0.03, 0.05)	diag(0.008, 0.008, 0.015)	Use `ams_np` values
L1	0.20 m	0.25 m	Use `ams_np` value
L2	0.20 m	0.20 m	Already matches
Mount offset	(0,0,−0.05)	(0,0,−0.05)	Already matches
Mount rotation	Identity (implicit)	Explicit 3×3 matrix	Must add mount rotation
Link masses	0 (ignored)	0.15, 0.12 kg	Must add
Link inertias	0 (ignored)	Full 3×3	Must add
Link COM offsets	N/A	midpoint of each link	Must add

3.4 Forward Kinematics Alignment

Current `mpc_ef`: Planar FK in body XZ plane, custom convention with −π joint-2 offset:

```
x_ee = L1*sin(q1) - L2*sin(q1+q2)
z_ee = ARM_MOUNT_Z - L1*cos(q1) + L2*cos(q1+q2)
y_ee = 0
```

ams_np: Full 3D DH FK with mount rotation:

```
R_0 = R_A @ mount_rotation
p_0 = p_A + R_A @ mount_offset
# then recursive: p[i+1] = p[i] + R[i] @ p_local[i]
```

Key differences:

- `ams_np` uses a non-trivial `mount_rotation` that maps platform → arm base. In the default model: arm base z-axis points along platform −y (right), arm base x-axis points along platform −z (down).
- Joint rotation is about z in DH frame = about −y in platform frame.
- The −π offset at joint 2 in `mpc_ef` is not present in `ams_np`; instead, the DH `alpha=0` and `a=link_length` naturally handle the geometry.

Action: Replace `forward_kinematics_body()` and `forward_kinematics()` with CasADi-symbolic versions of the `ams_np` DH FK. This also fixes the IK.

3.5 Inverse Kinematics Update

The current IK in `acados_solver.py` is derived from the old FK equations. After changing FK to match `ams_np`:

- Derive new geometric IK for the 2-link arm in the arm-base frame.
- Or use a numerical IK (Jacobian-based) that calls the new symbolic FK.
- This only matters for `cost_mode='ik'`; `cost_mode='ee'` bypasses IK entirely.

3.6 Frame Convention Mismatches (DANGER ZONES)

The `mpc_ef` and `ams_np` packages use **different frame conventions** in several places. Getting any of these wrong produces silently incorrect results (wrong signs, rotated axes). Each is a potential bug source.

3.6.1 Mount Rotation (HIGH RISK)

	mpc_ef	ams_np
Mount frame	**Implicit identity** — arm-base = platform frame	**Explicit rotation matrix** that maps platform → arm-base
Joint axis	Revolute about platform **Y-axis** (from URDF comment)	Revolute about arm-base **Z-axis** (DH convention)
"Down" direction	Platform **−Z** is down in arm frame	Arm-base **−X** is down (since x_0 = platform −z)

The `ams_np` mount rotation is:

```
mount_rotation = [[0, 1, 0],   # arm x₀ = platform y
                  [0, 0, -1],  # arm y₀ = platform −z
                  [-1, 0, 0]]  # arm z₀ = platform −x
```

This means:

- A joint rotation `θ` about arm-base z_0 = rotation about platform **−x**.
- `mpc_ef` treats joint rotation as about platform Y — **these are different axes.**
- Any FK, IK, or Jacobian expression that assumes joints rotate about platform Y will give wrong results with `ams_np` parameters.

Impact: Phases 1.3 (FK), 3.2 (IK), 4.2 (trajectory default positions). **Mitigation:** Always work in the arm-base DH frame throughout Phase 1. Convert to/from platform frame only at the interface (mount rotation). Validate FK output against `ams_np` at θ_1=90° to catch axis swap bugs.

3.6.2 Joint-2 Offset (MEDIUM RISK)

	`mpc_ef`	`ams_np`
Joint 2 zero config	−π offset in URDF → arm is **folded** at q_1=q_2=0	No offset → arm is **straight** at q_1=q_2=0
FK equation	`x = L1*sin(q1) − L2*sin(q1+q2)`	Standard DH: `a` parameter places link tip along x

This means the **same joint angle values** produce **different physical configurations** in the two conventions. If joint angle references or limits are carried over without adjusting for this offset, the arm will point in the wrong direction.

Impact: Phases 3.2 (IK initial guess), 4.2 (default arm config). **Mitigation:** In the new solver, define `arm_default_joints` based on the `ams_np` FK zero-config geometry, not the old `[0, π]` values. The test in Phase 0.2 should explicitly map equivalent configs.

3.6.3 Angular Velocity Frame (LOW RISK)

	`mpc_ef`	`ams_np`
`ω` in state vector	Described as "body frame" in docstring	World frame (all recursions are in world frame)
Euler's equation	Uses body-frame Euler: `I_body \ (τ − ω × I ω)`	Uses world-frame: `I_w = R I R^T`, then Newton-Euler in world

The `mpc_ef` model applies torques and computes `ω_dot` in body frame (diagonal inertia Euler equations). The `ams_np` dynamics works entirely in world frame. The state vector stores `ω` but interprets it differently.

In practice, the IMU callback in the controller stores `ω` from the IMU message, which is typically body-frame. If the MPC dynamics expects world-frame `ω`, a rotation mismatch arises.

Impact: Phase 1.6 (state derivative), Phase 4.1 (state estimator). **Mitigation:** The CasADi dynamics should follow `ams_np` convention (world-frame ω). In the controller, convert IMU body `ω` → world `ω` via `ω_world = R @ ω_body` before feeding to the solver. Add a unit test: spin platform at known ω in body, verify world-frame conversion matches.

3.6.4 Quaternion Convention (LOW RISK)

Both use `[qx, qy, qz, qw]` ordering and the same rotation matrix formula. **No mismatch** — but worth verifying in Phase 0.2.

Summary: Where Frame Bugs Will Bite

```
Phase 1.3 (casadi_fk)        ← mount rotation axis mapping
Phase 1.4 (casadi_kinematics)← ω frame (world vs body)
Phase 1.6 (state_derivative) ← thrust rotation, ω convention
Phase 3.2 (IK)               ← joint-2 offset, arm-base vs platform frame
Phase 4.1 (controller)       ← IMU ω body→world conversion
Phase 4.2 (trajectory)       ← default EE position, arm zero-config
```

---

4. Implementation Plan

> **Legend — task type annotations:**
>
> - **[MATH]** — requires deriving or verifying equations / physics (pen-and-paper or symbolic algebra).
> - **[MATH → CODE]** — translating known math into CasADi / Python (mechanical translation, but must get signs and frames right).
> - **[CODE]** — pure software engineering (wiring, config, testing, no new math).
> - **[TUNING]** — empirical parameter adjustment (no new derivations, but needs experiments).

> **Rule: NO existing files are modified.** Every change produces a new file. The old `mpc_ef` package is preserved untouched as a fallback.

Phase 0: Groundwork & Validation Harness — [CODE]

Goal: Set up testing infrastructure before changing anything. All tasks in this phase are pure code — no new math.

- ☐ **0.1** Create `ams_mpc_migration/test_dynamics.py`: *[CODE]*
  - Hover validation: ID at hover should give `F_z = m_total * g`, zero torques.
  - Free-fall validation: FD with zero inputs → `a = g`.
  - Round-trip: `FD(ID(qddot)) == qddot`.
  - Compare `ams_np` vs current `mpc_ef` predictions for identical states.
- ☐ **0.2** Create `ams_mpc_migration/test_fk.py`: *[CODE]*

  - Compare FK outputs between `mpc_ef` and `ams_np` at several joint configs.
  - Document the coordinate transform between the two conventions.
- ☐ **0.3** Snapshot current MPC performance: *[CODE]*
  - Log tracking errors, solve times, control effort for reference trajectories.
  - Save as baseline for comparison after migration.

Phase 1: CasADi Symbolic Dynamics Module — [MATH → CODE]

Goal: Translate `ams_np` coupled dynamics into CasADi symbolic expressions.

This phase is **not** deriving new math — the equations already exist in `ams_np`. The work is a **mechanical translation** from NumPy to CasADi SX. However, every line must respect frame conventions (see §3.6), making it error-prone. Validate each sub-module numerically against `ams_np` before proceeding.

- ☐ **1.1** Create `ams_mpc_migration/casadi_model_params.py`: *[CODE]*
  - Import `AerialManipulatorModel` parameters as constants.
  - Define `MASS`, `HOVER_THRUST`, `GRAVITY`, link params, etc.
  - Pure data extraction — no math.
- ☐ **1.2** Create `ams_mpc_migration/casadi_math_utils.py`: *[MATH → CODE]*
  - CasADi SX versions of `quat_to_rotation_matrix`, `quat_derivative`, `cross`, `skew`.
  - Direct translation from `ams_np/math_utils.py`.
- ☐ **1.3** Create `ams_mpc_migration/casadi_fk.py`: *[MATH → CODE]*
  - Implement `forward_kinematics_sym(q_A, p_A, theta)` in CasADi SX.
  - Use **Option (A)** approach: translate the recursive FK from `ams_np/kinematics.py` line-by-line into CasADi. Python `for` loops over the 2 joints are fine — CasADi unrolls them at graph-build time.
  - Returns CasADi symbolic: `R_list`, `p_list`, `p_com_list`.
  - Validate: evaluate numerically and compare against `ams_np.kinematics.forward_kinematics` at test configurations.
- ☐ **1.4** Create `ams_mpc_migration/casadi_kinematics.py`: *[MATH → CODE]*
  - CasADi SX versions of `velocity_recursion` and `acceleration_recursion` from `ams_np/kinematics.py`.
  - Same loop structure as NumPy version, but with CasADi symbolic types.
- ☐ **1.5** Create `ams_mpc_migration/casadi_dynamics.py`: *[MATH → CODE]*
  - CasADi SX version of `backward_recursion` and `forward_dynamics` from `ams_np/dynamics.py`.
  - For `forward_dynamics`: build the mass matrix column-by-column (same technique as `ams_np`), then `casadi.solve(M, u - h)` for the symbolic linear solve.
  - Validate: evaluate at test states and compare against `ams_np.dynamics.forward_dynamics`.
- ☐ **1.6** Create `ams_mpc_migration/casadi_state_derivative.py`: *[MATH → CODE]*

- Combine FK, velocity kinematics, and dynamics into a single function: `f(x, u) → x_dot` as a CasADi expression.
- Map control input `u = [thrust, τ_x, τ_y, τ_z, τ_j1, τ_j2]` to the `ams_np` input convention `[F_ext(3), τ_ext(3), τ_joint(2)]`.
- This mapping involves: `F_ext = R_A @ [0,0,thrust] + m_total * g_vec` — a frame rotation, not new physics.
- Include quaternion derivative: `q_dot = 0.5 * Ω(ω) @ q` (from 1.2).
- Validate: compare `casadi.Function` evaluation against `ams_np.simulator.state_derivative` at matching states.

Phase 2: New Acados Model — [CODE]

Goal: Create a new acados model file using the Phase 1 CasADi dynamics. No new math — this is wiring the symbolic expressions into acados structures. **No existing files are modified.**

- ☐ **2.1** Create `mpc_ef/acados_model_coupled.py` (NEW file): *[CODE]*
  - `export_quadrotor_model_coupled()` returns an `AcadosModel` with:
    - Same state vector `x` (17 states).
    - Updated `u`: last two entries are now joint torques, not accelerations.
    - `f_expl_expr` = the CasADi state derivative from Phase 1.
    - Includes `forward_kinematics()`, `forward_kinematics_body()`, `compute_ee_from_state()` — all new implementations using the `ams_np` DH convention.
    - `get_state_bounds()` with bounds matching `ams_np` parameters.
    - `get_control_bounds()` with torque limits instead of accel limits, thrust bounds recalculated from new `HOVER_THRUST`.
    - Updated constants: `MASS`, `HOVER_THRUST`, link lengths, etc.

Phase 3: New Acados Solver — [CODE + MATH]

Goal: Create a new solver file that uses the coupled model. **No existing files are modified.**

- ☐ **3.1** Create `mpc_ef/acados_solver_coupled.py` (NEW file): *[CODE]*
  - New `AcadosMPCSolverCoupled` class, based on the structure of the existing `AcadosMPCSolver` but importing from `acados_model_coupled`.
  - Updated default cost weights — arm control cost `R_arm` now penalizes torque (different units/magnitude than acceleration).
  - `cost_mode='ee'`: use new FK symbolic expressions from the coupled model.
  - Consider switching integrator to `IRK` (implicit Runge-Kutta) since coupled dynamics can be stiff.
- ☐ **3.2** Implement `inverse_kinematics_body()` in the new solver: *[MATH]*
  - **This is actual new math:** derive geometric IK for the `ams_np` arm in the arm-base DH frame (z-axis rotation, DH convention).
  - Must account for mount rotation when converting between platform body frame and arm-base frame.
  - Only needed for `cost_mode='ik'`; `cost_mode='ee'` bypasses IK.

- [ ] **3.3** Implement `compute_ee_reference()` in the new solver: *[CODE]*
  - Use new FK convention for body-to-world EE transform.
- [ ] **3.4** Solver tuning notes: *[TUNING]*
  - Re-tune `Q`, `R` weights.
  - Adjust SQP_RTI vs. full SQP.
  - Check solve time.

Phase 4: New Controller Node — [CODE]

Goal: Create a new controller script for the coupled MPC. **No existing files are modified.**

- [ ] **4.1** Create `scripts/mpc_controller_ef_coupled.py` (NEW file): *[CODE]*
  - Copy structure from `mpc_controller_ef.py`.
  - Import from `acados_solver_coupled` / `acados_model_coupled`.
  - The MPC now outputs `τ_joint` instead of `ddq`. For arm commands:
    - **(a)** Send predicted joint positions from MPC trajectory (`x_pred[1, 13:15]`) — works with position-controlled arm.
    - **(b)** Or send torques directly if using effort-based arm controller.
- [ ] **4.2** Update trajectory generation in the new controller: *[CODE]*
  - `get_ee_trajectory_point()`: EE default position calculation uses new FK with `ams_np` link lengths and mount rotation.
  - `ee_offset_z` parameter recalibrated for new link lengths.
- [ ] **4.3** Update logging and plotting in the new controller: *[CODE]*
  - Control labels: `ddq1, ddq2` → `τ_j1, τ_j2`.
  - Add logging of coupling forces if desired.

Phase 5: Validation & Tuning — [CODE + TUNING]

- [ ] **5.1** Hover test: *[TUNING]*
  - Deploy with `trajectory_mode='hover'` using the new `mpc_controller_ef_coupled.py`.
  - Verify: base stays at altitude, arm doesn't drift, low control effort.
  - Compare solve times against Phase 0 baseline.
- [ ] **5.2** Circle EE trajectory test: *[TUNING]*
  - Run `circle` mode. Observe:
    - EE tracking error (should improve vs. old model at moderate speeds).
    - Base disturbance rejection (the MPC now predicts arm reaction forces).
    - Yaw behavior (mount rotation may change the arm plane orientation).
- [ ] **5.3** Aggressive motion test: *[TUNING]*

    - Run `reach` or `line` at higher speed.
    - This is where coupling matters most — the old model would show base oscillations that the MPC couldn't anticipate.

- ☐ **5.4** Benchmark: *[CODE]*

    - Collect solve time statistics.
    - If solve time exceeds control period:
        - Try `SQP_RTI` with 1 iteration.
        - Reduce `N_horizon`.
        - Enable acados code generation.

---

5. File Change Map

All listed files are NEW. No existing files are modified or renamed.

```
am_description/
├── ams_np/                              # UNTOUCHED (reference
implementation)
│   ├── model.py
│   ├── kinematics.py
│   ├── dynamics.py
│   ├── math_utils.py
│   ├── state.py
│   └── simulator.py
│
├── ams_mpc_migration/                   # NEW — CasADi translation + tests
│   ├── __init__.py
│   ├── MIGRATION_PLAN.md                # This document
│   ├── casadi_model_params.py           # [CODE]     Model constants from
ams_np
│   ├── casadi_math_utils.py             # [MATH→CODE] CasADi
quat/cross/skew
│   ├── casadi_fk.py                     # [MATH→CODE] CasADi symbolic FK
│   ├── casadi_kinematics.py             # [MATH→CODE] CasADi
velocity/accel recursion
│   ├── casadi_dynamics.py               # [MATH→CODE] CasADi Newton-Euler
ID/FD
│   ├── casadi_state_derivative.py       # [MATH→CODE] Full x_dot = f(x,u)
│   ├── test_fk.py                       # [CODE]     Validation: ams_np vs
casadi FK
│   └── test_dynamics.py                 # [CODE]     Validation: ams_np vs
casadi dyn
│
├── mpc_ef/                              # UNTOUCHED (old files preserved
as-is)
│   ├── __init__.py                      #   (unchanged)
│   ├── acados_model.py                  #   (unchanged — old simplified
model)
│   ├── acados_solver.py                 #   (unchanged — old simplified
solver)
```

```
|   ├── acados_model_coupled.py          # NEW — coupled dynamics acados
model
|   ├── acados_solver_coupled.py         # NEW — coupled dynamics acados
solver
|   └── README.md                        #   (unchanged)
|
scripts/
|   ├── mpc_controller_ef.py             #   (unchanged — old controller)
|   └── mpc_controller_ef_coupled.py     # NEW — controller using coupled
MPC
```

---

6. Risk Assessment

Risk	Likelihood	Impact	Mitigation
CasADi symbolic dynamics too slow for real-time	Medium	High	Profile early; fall back to simplified coupling terms (Option D)
Acados code generation fails with complex expressions	Low	High	Simplify by pre-computing sub-expressions; use `casadi.Function` intermediate
Parameter mismatch with Gazebo URDF	Medium	Medium	Cross-check all values against URDF; add parameter loading from YAML
IK singularities at new arm lengths	Low	Low	Add numerical fallback IK; `cost_mode='ee'` avoids IK entirely
Stiff dynamics cause integrator issues	Medium	Medium	Switch from ERK to IRK integrator in acados options

---

7. Recommended Execution Order

```
Phase 0  (0.5 day)   [CODE]       Validation harness + baseline
measurements
    |
Phase 1  (2-3 days)  [MATH→CODE]  CasADi symbolic dynamics (Option A
recursive)
    |                               └─ mechanical translation, but ~5 frame-
convention
    |                                  danger zones require careful
attention (§3.6)
    |
Phase 2  (1 day)     [CODE]       New acados model file (wiring)
    |
Phase 3  (1 day)     [CODE+MATH]  New solver file + IK derivation (§3.2 is
actual math)
    |
Phase 4  (0.5 day)   [CODE]       New controller script
```

```
     |
Phase 5  (1-2 days)  [TUNING]     Validation, tuning, benchmarking
```

Total estimated effort: 5-8 days

Only **Phase 3 step 3.2** (IK derivation for new FK convention) requires new math. Everything else is either pure code or mechanical translation of already-validated `ams_np` equations.

---

8. Quick-Start: Minimal Viable Migration

If you want the fastest path to get coupled dynamics running:

1. **Skip IK entirely** — use `cost_mode='ee'` (already implemented). This removes the need to re-derive IK for the new FK convention.
2. **Recursive symbolic dynamics (Option A)** — translate the `ams_np` Newton-Euler recursion line-by-line into CasADi SX. Python `for` loops over i=0,1 are fine (CasADi unrolls them). Validate each sub-module against `ams_np` before combining.
3. **Keep `ddq` as control initially** if you want to defer the torque interface change. Compute `x_dot` as: given `ddq` as input, solve the coupled EOM for the resulting base accelerations. This preserves the current 6-control interface but still captures coupling in the base dynamics prediction. Switch to torque control later in a separate step.

---

9. Reference: Key Equations from `ams_np`

Forward Dynamics (what the MPC prediction model needs)

Given state `(p, v, q, ω, θ, θ̇)` and input `(F, τ, τ_joint)`:

```
        ⎡          ⎤   ⎡                  ⎤   ⎡              ⎤
        ⎢  a_base  ⎥   ⎢  F_ext            ⎥   ⎢  h_base(q,q̇) ⎥
M(q)    ⎢  α_base  ⎥ = ⎢  τ_ext            ⎥ - ⎢  h_rot(q,q̇)  ⎥
        ⎢  θ̈       ⎥   ⎢  τ_joint          ⎥   ⎢  h_arm(q,q̇)  ⎥
        ⎣          ⎦   ⎣                  ⎦   ⎣              ⎦
```

Where `M(q)` is the 8×8 coupled mass matrix and `h` contains Coriolis, centrifugal, and gravitational terms.

State derivative

```
x_dot = [ v,  a_base,  0.5*Ω(ω)*q,  α_base,  θ̈,  θ̇ ]
```

This is the expression that needs to be in CasADi SX for acados.